AI Exam Answer Grader

A Reusable NLP Asset for Rubric-Based Descriptive Answer Assessment

Week 7 Deliverable — Integration with Group 1, a Deployable Web Application and Responsible AI | Target: 17 – 23 Oct 2026

Group 2: E008 Ritwik Shetty, E009 Chaitanya Thakur, E010 Dhruvit Thakkar, E011 Ajinkya Hatle, E012 Bharat Baswan

1. Summary

At the mid-term evaluation ma'am asked how our grader would work on Group 1's data, which differs from the Mohler answers in every way that matters: it is handwriting read by OCR, answers fill whole pages, the subject is computer networks, and the digitizer returns one record per page with the student's name at the top. Week 7 answers this question with Group 1's real output and with measurements, and replaces the Streamlit demo with a Django + React web application.

Main results: the grader now accepts Group 1's records exactly as their digitize_page() returns them (a new adapter and two new API endpoints), and the full chain photo → Group 1 → our grader → teacher works in the new web application. At Group 1's neat-handwriting error rate (CER 0.04) OCR barely changes our marks (MAE 0.567 → 0.659 marks out of 5); at their average and cursive rates (CER 0.19 and 0.34) the error doubles (1.15 and 1.34 marks) and only 47% and 34% of marks stay within ±1 mark. The similarity model is hurt about twice as much as the LLM. Our own review rules catch only part of this, so Group 1's OCR confidence is used to send low-quality pages to a teacher, who can correct the text before grading. An OCR-aware LLM prompt was tested and did not help, so it is off by default. On Group 1's real pages the marks were too generous for a new course (about 8 / 10 for pages that cover at most 3 of 7 rubric points), because our calibration was learned from the lenient Mohler teachers; such long answers are now flagged for a teacher.

2. The Question from the Mid-Term Evaluation

Ma'am asked how our grader would work on Group 1's data, since it is so different from the Mohler answers we trained and tested on. We ran Group 1's own digitizer on the real answer sheets in their repository and compared its output with our data:

	Mohler (our training and test data)	Group 1's digitizer output
Text	Typed, no errors	Read by OCR from handwriting: CER 0.04 on a neat page, 0.35 on a cursive page
Length	19 words on average; 95% have 45 words or fewer	140–190 words on one page, and answers continue over several pages
Subject	Data structures and C++ (79 questions)	Computer networks (the OSI model and email)
Unit	One answer per question	One record per page; name, roll number and date at the top
Student ID	Always given	Often unreadable (E007 read as "Eool", E013 as "Ford")
Confidence field	confidence (course handout format)	ocr_confidence
Teacher marks	Two teachers per answer	None

We answered the question in three ways: (1) an adapter so that the grader accepts Group 1's records exactly as they are, (2) grading Group 1's two real pages from the true text and from the OCR text, and (3) a controlled experiment that measures, on answers with teacher marks, how much OCR errors at Group 1's real error rates change our marks. Group 1's code was used unchanged.

3. Group 1's Digitizer on Their Real Pages

Group 1's reusable function digitize_page() (Tesseract OCR with their Week 7 image cleaning, plus rule-based NER) took about one second per page. Against their hand-typed transcriptions:

Page	Handwriting	True student ID	CER	WER	OCR confidence
aayush_page-01	neat	E007	0.038	0.163	0.82
rish_page-1	cursive	E013	0.348	0.974	0.4

CER and WER = share of characters and words that are wrong (lower is better). These match Group 1's own figures.

On the neat page almost every word is right; on the cursive page about one character in three is wrong, but Tesseract's own confidence (0.40) warns about it. On both pages the question number was found ("QL" and "@1]" were read as Q1), but the student ID was not.

4. The Adapter: from Pages to Answers

A new module, `group1.py`, turns Group 1's page records into answers the grader can mark. It does what a teacher would do by eye:

1. Finds the question number at the start of a line, including OCR mistakes ("QL.", "@1]", "Q.1" → Q1).
2. Removes the header above the first question (name, roll number, date), so it is not graded as part of the answer.
3. Splits a page that holds two questions, and joins pages when an answer continues on the next page (a page with no question number continues the previous answer).
4. Keeps the lowest OCR confidence of the pages an answer came from, so one bad page is enough to flag it.
5. Uses the student ID typed by the teacher when the digitizer could not read it, and adds a review note when the ID or the question number is missing.
6. Calls Group 1's digitize_page() from their project folder (set with GROUP1_DIGITIZER_PATH), so a scanned image can go straight into our service.

Example from the neat page: the header "ISP Assignment % - Eool hayush Nayak" was removed, the answer starting "QL OS! Model in Email Sending and Recening…" was labelled Q1, and the note "The student ID could not be read from the answer sheet" was added for the teacher.

5. Grading Group 1's Real Pages

Group 1's pages answer an Internet Security assignment question about the OSI model and email. Group 1 has no answer key or teacher marks, so we wrote a reference answer and a 7-point rubric ourselves (one point per layer group, 10 marks). Only page 1 of each answer sheet was scanned, so each page covers only part of the answer. Each page was graded from Group 1's true text, from the OCR text, and from the OCR text with the new OCR-aware prompt (Section 6):

Page	Graded text	Mark /10	Similarity	LLM	Rubric	Points present / partial	Flagged
aayush_page-01	true text	8.0	3.86	2.5	1.5	1 / 3	True
aayush_page-01	OCR text	8.0	3.71	2.5	1.5	1 / 3	True
aayush_page-01	OCR text + OCR-aware prompt	8.0	3.71	2.5	1.5	1 / 3	True
rish_page-1	true text	8.5	4.07	3.0	2.0	3 / 0	True
rish_page-1	OCR text	7.5	2.75	5.0	2.0	3 / 0	True
rish_page-1	OCR text + OCR-aware prompt	7.0	2.75	4.5	2.0	3 / 0	True

Similarity, LLM and rubric scores are on a 0–5 scale. Rubric: 7 points. All six gradings were flagged for a teacher.

On the neat page (CER 0.04) the OCR text got exactly the same mark as the true text (8 / 10), with the same rubric points. On the cursive page (CER 0.35) the similarity score fell from 4.07 to 2.75 and the mark from 8.5 to 7.5. That answer was sent to a teacher for four reasons at once: low OCR confidence (0.40), low confidence (0.51), a 3-mark gap between the LLM and the rubric score, and its length.

The marks are too generous for this course. Each page covers at most 3 of the 7 rubric points, yet every version got 7 / 10 or more. There are two causes. The calibration was learned from the lenient Mohler teachers and maps even a low score upwards (on the neat page, an LLM score of 2.5 and a rubric score of 1.5 out of 5 became 4.1). And the similarity model was trained on short data-structures answers and reads only about half of a page-long answer. All six gradings were flagged, so a teacher would see them; recalibrating on a few teacher-marked answers from the new course is the fix (Section 12).

The LLM also listed OCR garbage as wrong statements (for example "Ginall Outlook" for "Gmail, Outlook"), which costs half a rubric point each. The OCR-aware prompt did not stop this.

6. Controlled Experiment: How Much Do OCR Errors Change Our Marks?

Group 1's pages have no teacher marks, so to measure accuracy under OCR errors we added OCR-style errors to Mohler answers, which do have marks. The errors copy what Tesseract did on Group 1's pages: look-alike letters ("the" → "bhe", "layer" → "layer", "m" → "rn"), missing letters, stray punctuation, and words split or joined. The error rates were set to Group 1's measured character error rates: 0.04 (neat page), 0.19 (their average) and 0.34 (cursive page).

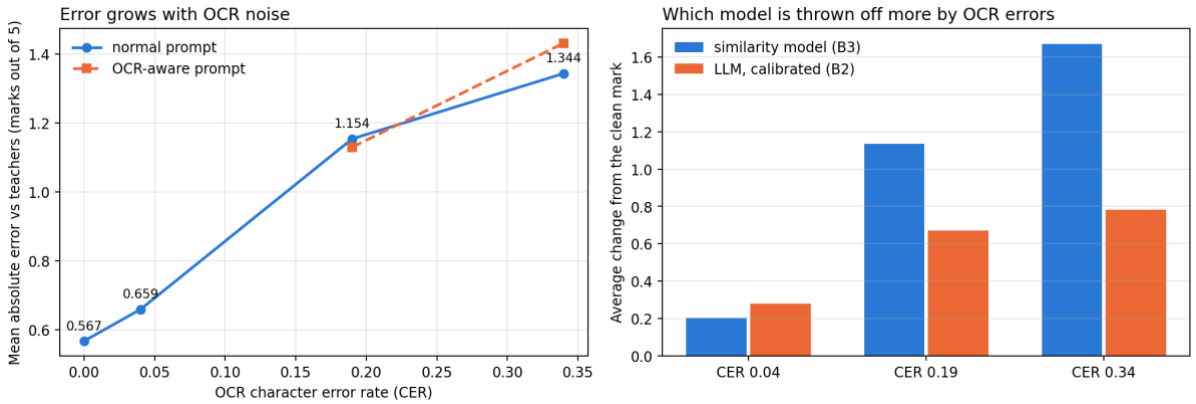
We used the 95 fold-0 answers of the Week 5 evaluation sample with their sampling weights, and retrained the similarity model on folds 1–4 so that it had never seen them. Each answer was graded clean, at each error rate, and at the two higher rates with an OCR-aware prompt that tells the LLM the text came from OCR: 570 gradings in total. The same noisy text was used for both prompts, so they can be compared answer by answer.

Condition	CER	MAE vs teachers	Within ±1 (%)	Mark change vs clean	Similarity change / LLM change	Flagged (%)
clean	0.0	0.567	75.6	0.0	0.0 / 0.0	23.1
CER 0.04	0.04	0.659	76.0	0.222	0.203 / 0.28	17.8
CER 0.19	0.19	1.154	47.2	0.847	1.134 / 0.67	39.0
CER 0.34	0.34	1.344	33.9	1.174	1.67 / 0.781	37.9
CER 0.19 + OCR-aware prompt	0.19	1.13	45.8	0.835	1.134 / 0.72	42.8
CER 0.34 + OCR-aware prompt	0.34	1.432	22.2	1.237	1.67 / 0.891	43.7

95 fold-0 answers from the Week 5 sample, weighted; marks out of 5; the similarity model was retrained without fold 0. "Change" = average absolute difference from the same answer without OCR errors.

Comparison	MAE gain	95% CI	Verdict
CER 0.19 + OCR-aware prompt vs CER 0.19	0.024	[-0.099, 0.131]	not significant
CER 0.34 + OCR-aware prompt vs CER 0.34	-0.088	[-0.199, 0.029]	not significant
clean vs CER 0.19	0.587	[0.132, 0.907]	real difference
clean vs CER 0.34	0.777	[0.258, 1.117]	real difference

Bootstrap over questions (2,000 resamples). A positive gain means the first condition has the lower error.



Left: error against the teachers as OCR noise grows. Right: how much each model's score moves away from its clean value.

Neat handwriting barely matters: at CER 0.04 the error against the teachers rises only from 0.567 to 0.659 marks, ±1 agreement is unchanged (76%), and marks move by 0.22 on average.

Messy handwriting matters a lot: at CER 0.19 the error doubles (1.154) and at 0.34 it reaches 1.344; only 47% and 34% of marks stay within ±1 mark. Both drops are significant.

The similarity model is hurt about twice as much as the LLM (its score moves by 1.13 and 1.67 marks at CER 0.19 and 0.34, the LLM's by 0.67 and 0.78): character errors break its word matching, while the LLM still reads the words the student meant.

Our own review rules catch only part of the damage: 38–39% of the noisy answers are flagged, against 23% of the clean ones. OCR confidence is therefore the key safeguard. Group 1's digitizer gave 0.82 for the neat page and 0.40 for the cursive one, and every answer below 0.80 goes to a teacher, who can correct the transcription in the web application before grading.

The OCR-aware prompt did not help (+0.024 marks at CER 0.19 and −0.088 at 0.34, neither significant), so it is kept as an option (from_ocr=True) but switched off by default.

These results come from 95 answers with simulated errors and two real pages, so they show the size of the effects rather than exact rates for Group 1's students.

7. Long Answers

The similarity model reads at most 256 word-pieces of the reference answer and the student answer together. Group 1's pages are 140–190 words, so with our reference answer the model sees only about the first half of each page; the LLM reads all of it. Only 6 of the 2,264 Mohler answers (0.3%) are longer than 100 words, so the model was never trained on answers this long. Week 7 therefore adds a review rule: an answer of more than 100 words is flagged ("Long answer: much longer than the answers the similarity model was trained on, so check the mark"). It affects very few Mohler answers but every full page from Group 1.

8. New API Endpoints

Two endpoints were added to the FastAPI service; the four Week 6 endpoints are unchanged.

Endpoint	Input	Output
`POST /digitize`	One page image (.jpg or .png) and an optional page number (multipart form)	Group 1's record: student_id, question, answer_text, page, marks_detected, ocr_confidence, entities_found
`POST /evaluate-group1-pages`	answer_key, pages (Group 1's records exactly as returned) and an optional student_id typed by the teacher	One result per answer, as in /evaluate-batch, plus pages, the graded answer_text and the removed header
`GET /health (updated)`	–	Also reports whether Group 1's digitizer is installed

Example: Group 1 → Group 2 in Python

```
import requests
from digitizer import digitize_page          # Group 1's asset

page = digitize_page("answer sheet page 1.jpg", page_number=1)
reply = requests.post("http://localhost:8000/evaluate-group1-pages",
                      json={"answer_key": answer_key, "pages": [page], "student_id": "E007"})
for result in reply.json()["results"]:
    print(result["question"], result["score"], result["maximum_score"], result["needs_human_review"])
```

Recorded result for the neat page: **8 / 10**, confidence 0.86, pages [1], flagged for review: 1 reason(s).

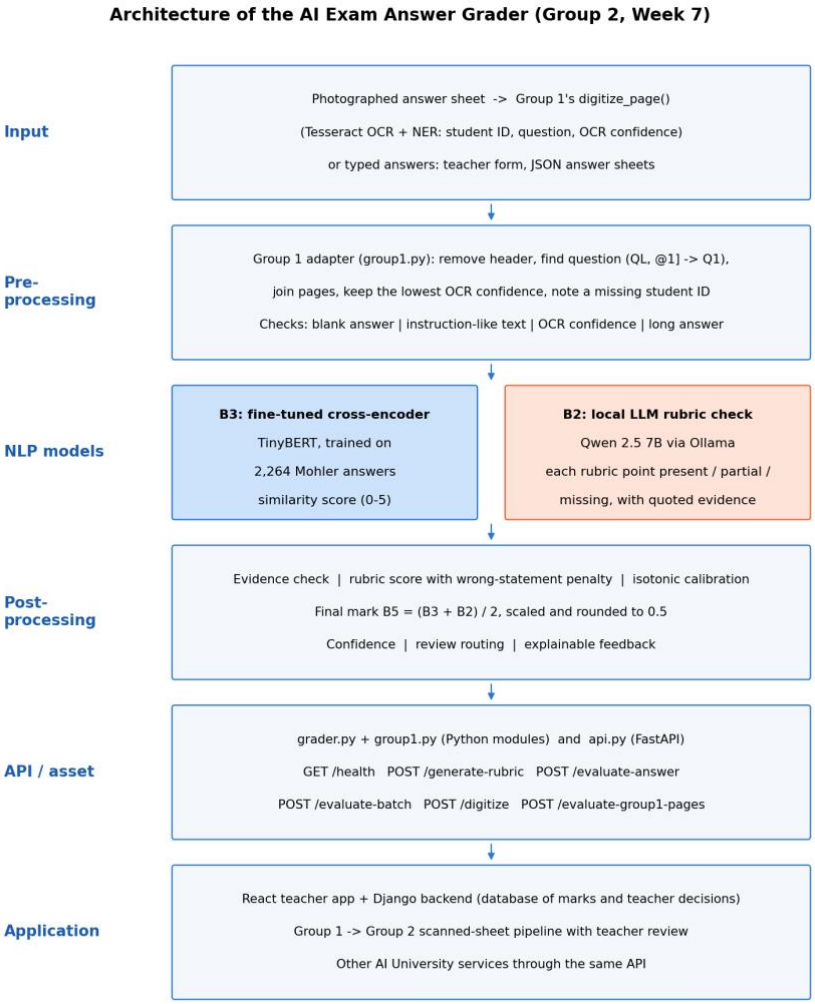
Wrong requests get clear errors: an answer whose question is not in the answer key returns HTTP 400 with the question number and pages, a page without answer_text returns 422, and a file that is not an image returns 400.

9. A Deployable Web Application: Django + React

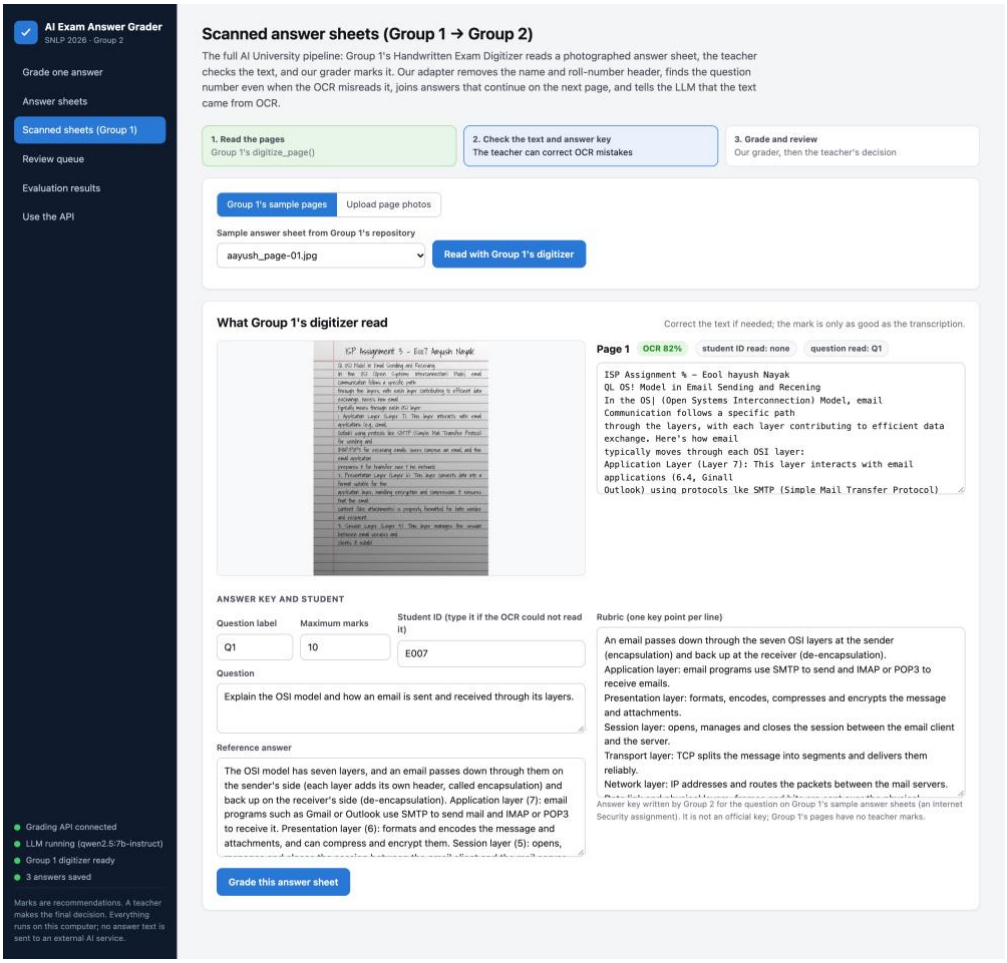
Streamlit was good for a prototype but is limited for deployment: it reruns the whole script on every click, keeps no records, and cannot easily serve many users. Week 7 replaces it with a standard web stack: a React frontend for the teacher interface and a Django backend that stores every graded answer and every teacher decision in a database. Django calls our FastAPI grading service over HTTP exactly as any other application would, which also demonstrates that the asset is reusable.

New compared with the Streamlit demo: a review queue of flagged answers waiting for a teacher, teacher decisions saved permanently with a comment and the time, a CSV export of all marks, the scanned-sheet page for Group 1 (upload photos, check and correct the OCR text, grade), and Django's admin pages for staff. It runs with three commands: Ollama, uvicorn api:app --port 8000, and python manage.py runserver 8001 in web/backend (the React app is built once with npm run build).

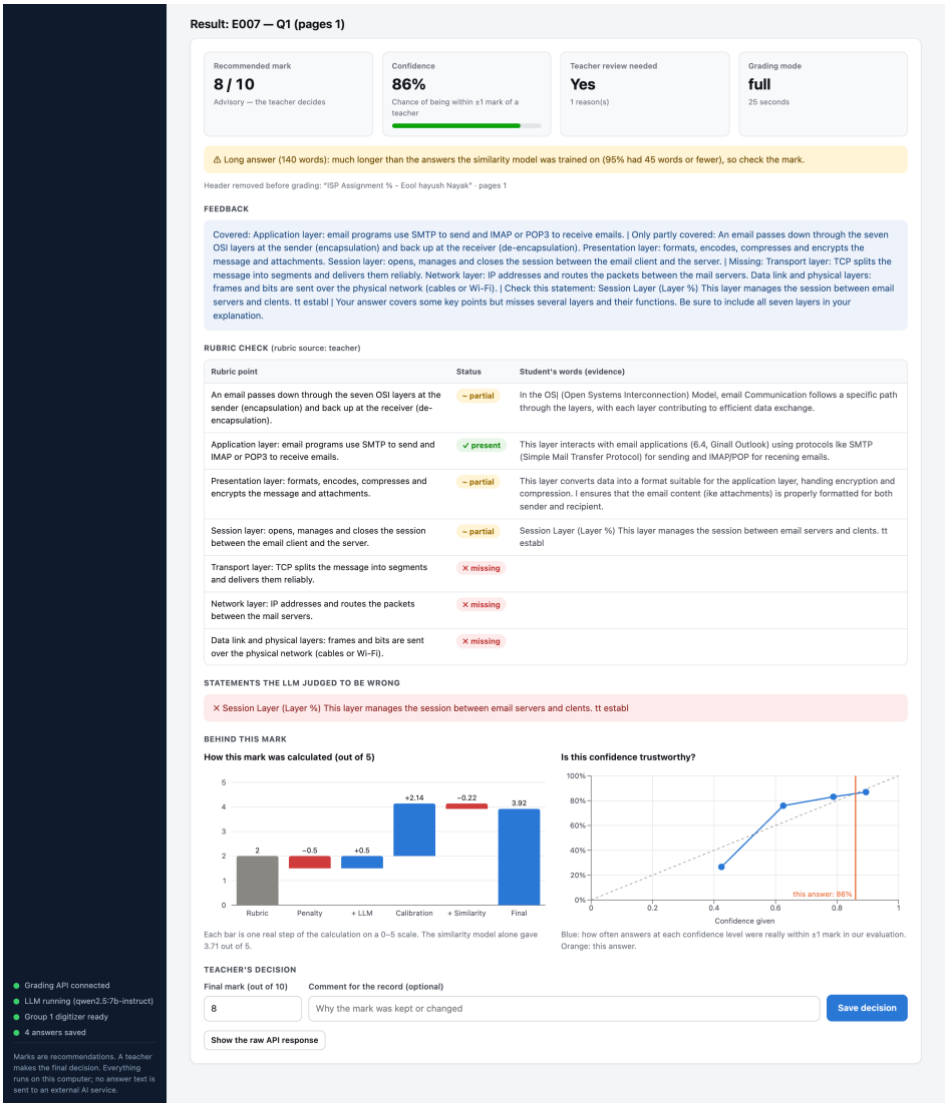
Part	What it does
React frontend (web/frontend)	Six pages: grade one answer, answer sheets, scanned sheets (Group 1), review queue, evaluation results, use the API. Charts with Recharts. Built with Vite into static files.
Django backend (web/backend)	Serves the React app, saves every graded answer and every teacher decision in a database (SQLite), exports CSV, and calls the FastAPI grading service like any other client. Django admin lets staff manage exams and answers.
FastAPI grading service (api.py)	Unchanged reusable asset: grader.py (cross-encoder + local LLM) and group1.py (adapter + Group 1's digitizer).



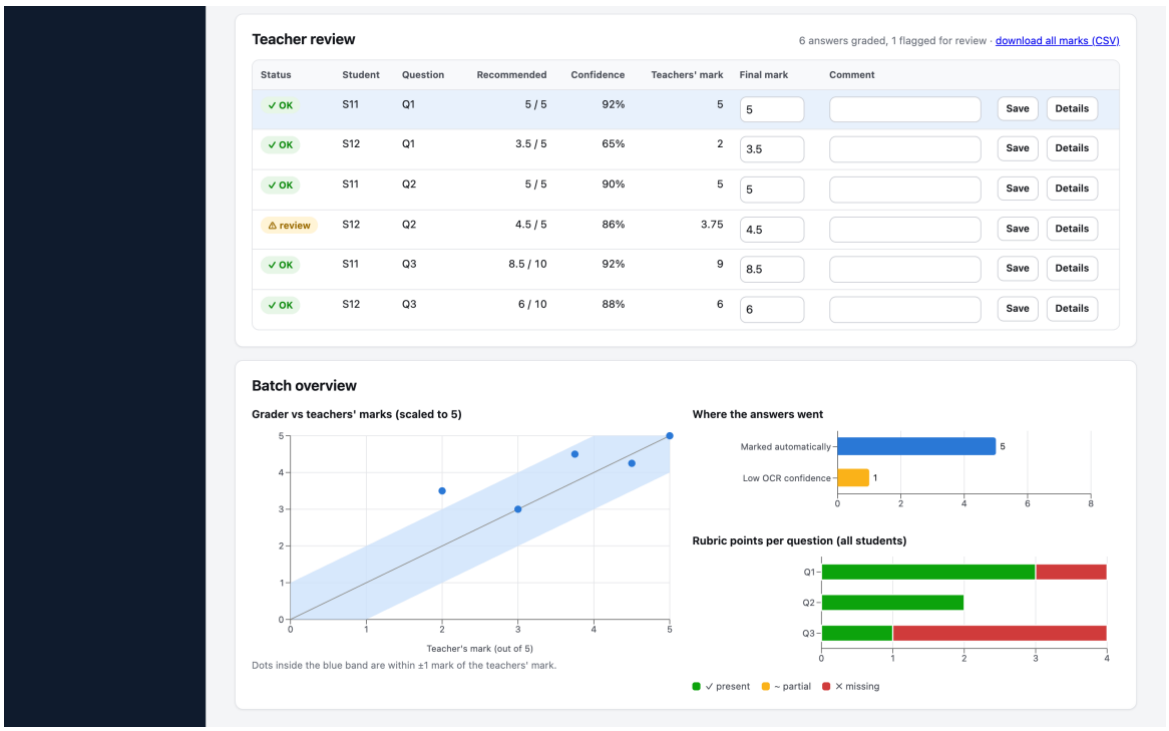
Architecture after Week 7: Group 1's digitizer at the input, the adapter in pre-processing, two new endpoints, and the React + Django teacher application.



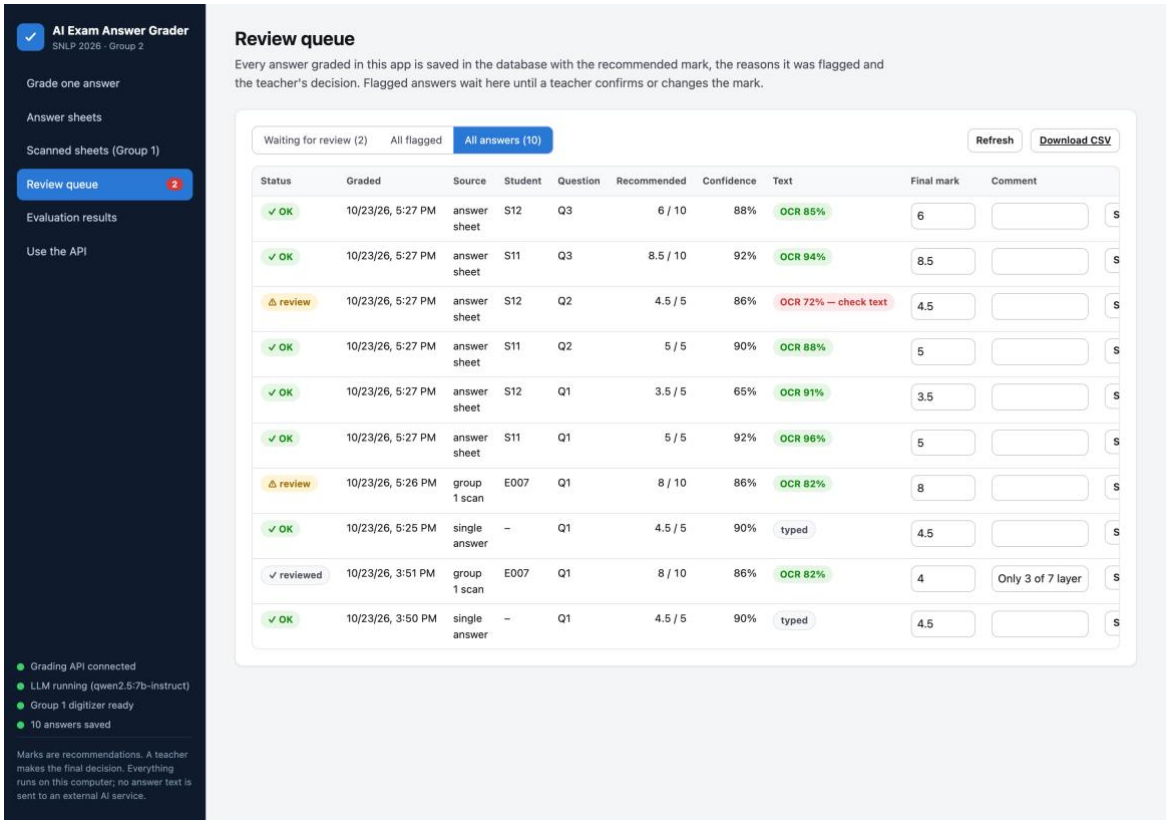
Scanned sheets (Group 1): the photographed page, the text read by Group 1's digitizer with its OCR confidence, and the answer key. The teacher can correct the text before grading.



The graded scanned page: mark, confidence, review reasons (here: long answer), the removed header, the rubric check with the student's own words, and how the mark was calculated.



Answer sheets in Group 1's format with OCR confidence: the teacher review table (one answer flagged, partly for low OCR confidence) and the batch overview.



Review queue: every graded answer is stored with the recommended mark, the teacher's final mark and comment, and can be exported as CSV.

10. Integration with the AI University

The scanned-sheet page shows the full chain that the handout describes for Group 1 → Group 2: a photographed answer sheet → Group 1's digitize_page() → our adapter → our grader → the teacher's review, with every step visible and the teacher able to correct the OCR text before grading. The same chain

is available to any program through POST /digitize and POST /evaluate-group1-pages. Group 12's malicious-input detector could sit between the digitizer and our LLM in the same way.

11. Responsible AI

- **Privacy:** Everything still runs on the local machine: Group 1's OCR, our models and the LLM. Uploaded page photos are processed and deleted; the database keeps only text and marks. Real answer sheets contain names and roll numbers, so the database must stay on a college computer. The header with the student's name is removed before grading, so it never reaches the LLM prompt.
- **Bias:** OCR errors depend on handwriting: the cursive page had nine times the character error rate of the neat one, and our measurements show this can change marks by more than a mark. Without safeguards, students with untidy handwriting would be penalised for their handwriting rather than their knowledge. Low OCR confidence therefore always sends the answer to a teacher, who can correct the text before grading.
- **Hallucination:** The LLM sometimes treats OCR garbage as a wrong statement ("Ginall Outlook"), or overrates garbled text: it gave 5 out of 5 to the cursive OCR text while its own rubric check gave 2. The evidence check and the LLM–rubric gap rule flag such cases.
- **Explainability:** Each mark comes with the rubric points covered, the student's own words as evidence, any wrong statements, the removed header and a chart of how the mark was calculated, so a teacher can check it quickly.
- **Security:** Only .jpg and .png images are accepted and they are deleted after reading, Django's CSRF protection is on, and the prompt-injection defences from Week 6 also apply to OCR text. Teacher logins are not yet added (Section 12).
- **Human oversight:** The system never finalises a mark: flagged answers wait in the review queue, the teacher's mark and comment are stored next to the recommendation, and the CSV export uses the teacher's mark when there is one.

12. Limitations and Next Steps (Week 8)

- Only two real pages from Group 1, with no teacher marks; the OCR experiment uses simulated errors on 95 answers.
- The calibration and the similarity model come from one lenient course with short answers; on a new course with long answers the marks are too generous until the grader is recalibrated on a few teacher-marked answers.
- The similarity model reads only about the first 256 word-pieces; long answers are flagged but not yet split into parts.
- Group 1's digitizer misses most handwritten student IDs, so the teacher has to type them.
- The web application has no teacher login or roles yet, uses SQLite, and grades one answer at a time.

Week 8

- Final testing, including automated tests for the adapter and the API.
- The final repository in the recommended structure with a README, and the final report.
- A ten-slide presentation and a rehearsed demonstration of the Group 1 → Group 2 chain and the known failures.